7.3.5 Sending Data Messages

You can send binary data via SMS using the sendDataMessage method on an SMS Manager. The sendDataMessage method works much like sendTextMessage, but includes additional parameters for the destination port and an array of bytes that constitutes the data you want to send. The basic structure of sending a data message looks like this:

```
Intent sentIntent = new Intent(SENT_SMS_ACTION);
      PendingIntent sentPI = PendingIntent.getBroadcast(getApp
licationContext(),
                                                          0,
sentIntent, 0);

      short destinationPort = 80;
      byte[] data = [ ...  your data ... ];
      smsManager.sendDataMessage(sendTo, null, destinationPort,
                                 data, sentPI, null);
```

7.3.6 Listening for Incoming SMS Messages

When a new SMS message is received by the device, a new Broadcast Intent is fired with the android.provider.Telephony.SMS_RECEIVED action. Note that this is a string literal. The SDK currently doesn't include a reference to this string, so you must specify it explicitly when using it in your applications.

For an application to listen for SMS Broadcast Intent, it needs to specify the RECEIVE_SMS manifest permission. Request this permission by adding a <uses-permission> tag to the application manifest, as shown in the following snippet:

```
<uses-permission
android:name="android.permission.RECEIVE_SMS"
      />
```

The SMS Broadcast Intent includes the incoming SMS details. To extract the array of SmsMessage objects packaged within the SMS Broadcast Intent bundle, use the PDU extras key to extract an array of SMS PDUs (protocol description units—used to encapsulate an SMS message and its metadata), each of which represents an SMS message. To convert each PDU byte array into an SMS Message object, call SmsMessage.createFromPdu, passing in each byte array:

```
Bundle bundle = intent.getExtras();
     if (bundle != null) {
```